

PW EARNERS
DSA in C++

DPP — 3 (SOLUTIONS)

Topic: Loops (For, While, Do-While, Break & Continue)

For Q1 to Q12: Predict the exact output of each program block. If there is a compilation error, infinite loop, or undefined behavior, state the reason clearly.

Q1. (MCQ)

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     for(int i = 1; i <= 3; i++) {
5         cout << i << " ";
6     }
7     return 0;
8 }
```

Output:

1 2 3

Explanation: This is a standard for loop running from $i = 1$ up to $i = 3$. It executes three times and prints the values sequentially.

.....

Q2. (MCQ)

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 3;
5     while(x--) {
6         cout << x << " ";
7     }
8     return 0;
9 }
```

Output:

2 1 0

Explanation: The post-decrement operator ($x--$) uses the current value to evaluate the condition, then decreases it. When $x = 3$ (true), it drops to 2 and prints 2. This continues until $x = 0$ (false condition), where it drops to -1 and the loop terminates.

.....

Q3. (MCQ)

```
1 #include<iostream>
2 using namespace std;
3 int main(){
```

```

4     int count = 0;
5     do {
6         cout << "Loop ";
7         count++;
8     } while (count < 0);
9     return 0;
10 }

```

Output:

Loop

Explanation: A do-while loop guarantees at least one execution before evaluating the condition. It prints “Loop”, increments `count` to 1, and then terminates because $1 < 0$ is false.

Q4. (MCQ)

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     for(int i = 0; i < 5; i++) {
5         if(i == 2) break;
6         cout << i << " ";
7     }
8     return 0;
9 }

```

Output:

0 1

Explanation: When i reaches 2, the `break` statement is triggered. This immediately terminates the entire loop, skipping the printing of 2 and any subsequent iterations.

Q5. (MCQ)

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int sum = 0;
5     for(int i = 1; i <= 4; i++) {
6         if(i % 2 == 0) continue;
7         sum += i;
8     }
9     cout << sum;
10    return 0;
11 }

```

Output:

4

Explanation: The `continue` statement forces the loop to skip adding even numbers (2 and 4) to `sum`. Thus, only odd numbers (1 and 3) are accumulated ($1 + 3 = 4$).

Q6.

```

1 #include<iostream>

```

```

2 using namespace std;
3 int main(){
4     int i = 0;
5     while(i < 3) {
6         cout << i << " ";
7         i += 2;
8     }
9     return 0;
10 }

```

Output:

0 2

Explanation: Starts with $i = 0$ and increments by 2. It prints 0, increments to 2, prints 2, and increments to 4. The condition $4 < 3$ fails, ending the loop.

.....

Q7.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int i = 1;
5     for( ; i <= 5; ) {
6         cout << i << " ";
7         i = i + 2;
8     }
9     return 0;
10 }

```

Output:

1 3 5

Explanation: A for loop does not require initialization or update blocks in its signature. It behaves identically to a while loop here, starting at 1, printing, and increasing by 2 each time.

.....

Q8.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     for(int i = 1, j = 10; i < j; i++, j -= 3) {
5         cout << i << " ";
6     }
7     return 0;
8 }

```

Output:

1 2 3

Explanation: Iteration 1: $i = 1, j = 10$ (prints 1). Update: $i = 2, j = 7$.

Iteration 2: $i = 2, j = 7$ (prints 2). Update: $i = 3, j = 4$.

Iteration 3: $i = 3, j = 4$ (prints 3). Update: $i = 4, j = 1$.

Iteration 4: $4 < 1$ is false. Loop terminates.

.....

Q9.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int i = 0;
5     while(i = 3) {
6         cout << "PW ";
7         break;
8     }
9     return 0;
10 }

```

Output:

PW

Explanation: The single = is an assignment, not an equality check. `i = 3` sets `i` to 3, which is evaluated as true (non-zero). The loop enters, prints “PW ”, and instantly hits the `break`, ending execution.

.....

Q10.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     for(int i = 1; i <= 2; i++) {
5         for(int j = 1; j <= 3; j++) {
6             if(j == 2) break;
7             cout << i << " " << j << " ";
8         }
9     }
10    return 0;
11 }

```

Output:

1 1 2 1

Explanation: The inner loop always triggers a `break` when `j` reaches 2. Therefore, for each outer loop iteration (`i = 1` and `i = 2`), it only prints the values of `i` and `j` for the case where `j = 1`. This produces 1 1 followed by 2 1.

.....

Q11. (Crazy Question MCQ)

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     for(int i = 0; i < 5, false; i++) {
5         cout << "Hello ";
6     }
7     cout << "Done";
8     return 0;
9 }

```

Output:

Done

Explanation: The condition `i < 5, false` uses the comma operator. The comma operator evaluates the left side but only returns the rightmost value (`false`). Thus, the loop condition is always false and never executes.

.....

Q12. (Crazy Question MCQ)

```

1 #include <iostream>
2 using namespace std;
3 int main(){
4     int i = 0;
5     for(;;) {
6         if(i == 5) {
7             break;
8         }
9         cout << i << " ";
10        i += 2;
11    }
12    return 0;
13 }
```

Output:

Infinite Loop

Explanation: The loop counter starts at 0 and increments by 2 (0, 2, 4, 6...). The condition to break checks for exact equality with 5 (`i == 5`), which is entirely skipped. Therefore, the loop runs forever.

.....

For Q13 to Q30: Write C++ programs for the following. Ensure proper loop condition checks and variable initializations.

Q13. Take a positive integer N as input and compute the sum of all natural numbers from 1 up to N using a loop.

Sample 1 — Input: 5 $\Rightarrow$ Output: 15

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int N, sum = 0;
5     cin >> N;
6     for(int i = 1; i <= N; i++) {
7         sum += i;
8     }
9     cout << sum << endl;
10 }
```

Explanation: A simple for loop sequentially adds all values from 1 to N into the `sum` variable.

.....

Q14. Write a program that takes two integers as input and prints all the even numbers located strictly between them.

Sample 1 — Input: 3 10 $\Rightarrow$ Output: 4 6 8

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int a, b;
5     cin >> a >> b;
6
7     // Swap to ensure 'a' is the lower bound
8     if(a > b) {
9         int temp = a;
10        a = b;
11        b = temp;
12    }
13
14    for(int i = a + 1; i < b; i++) {
15        if(i % 2 == 0) cout << i << " ";
16    }
17 }

```

Explanation: We sort the bounds using a temporary variable just in case the user inputs the larger number first. Then, we iterate strictly between them ($a + 1$ to $b - 1$), printing numbers cleanly divisible by 2.

.....

Q15. Write a program to take a number as input and compute the product of all its individual digits.

Sample 1 — Input: 413 $\Rightarrow$ Output: 12

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n;
5     cin >> n;
6     if(n == 0) {
7         cout << 0 << endl;
8     } else {
9         int prod = 1;
10        while(n > 0) {
11            int lastDigit = n % 10;
12            prod *= lastDigit;
13            n /= 10;
14        }
15        cout << prod << endl;
16    }
17 }

```

Explanation: We first check if the number itself is 0 using an if-else structure. If it is positive, modulo 10 isolates the lastDigit. We multiply it to our product accumulator and integer-divide by 10 to remove that digit for the next iteration.

.....

Q16. Given a positive integer N , write a program to find and print all of its divisors.

Sample 1 — Input: 12 $\Rightarrow$ Output: 1 2 3 4 6 12

```

1 #include <iostream>

```

```

2 using namespace std;
3 int main() {
4     int N;
5     cin >> N;
6     for(int i = 1; i <= N; i++) {
7         if(N % i == 0) cout << i << " ";
8     }
9 }

```

Explanation: A divisor leaves a remainder of 0. We loop up to N and check divisibility using the modulo operator.

.....

Q17. Write a program to find if a given positive integer is a Perfect Number. A perfect number is a positive integer that is equal to the sum of its proper divisors.

Sample 1 — Input: 28 $\Rightarrow$ Output: Perfect Number

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n, sum = 0;
5     cin >> n;
6     for(int i = 1; i < n; i++) {
7         if(n % i == 0) sum += i;
8     }
9     if(sum == n) cout << "Perfect Number" << endl;
10    else cout << "Not a Perfect Number" << endl;
11 }

```

Explanation: We iterate up to $n - 1$ aggregating factors, then compare the total sum to the original number.

.....

Q18. Write a program to print all integers from 1 to 500 that are **not** divisible by 3. You must use the `continue` keyword.

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     for(int i = 1; i <= 500; i++) {
5         if(i % 3 == 0) continue;
6         cout << i << " ";
7     }
8 }

```

Explanation: The `continue` statement catches multiples of 3 and forces the loop to skip the print execution for those numbers, immediately moving to the next iteration.

.....

Q19. Take an integer N as input and print the Fibonacci series up to N terms. (0, 1, 1, 2, 3, 5, ...)

Sample — Input: 6 $\Rightarrow$ Output: 0 1 1 2 3 5

```

1 #include <iostream>
2 using namespace std;
3 int main() {

```

```

4     int n;
5     cin >> n;
6     int a = 0, b = 1;
7     for(int i = 1; i <= n; i++) {
8         cout << a << " ";
9         int nextTerm = a + b;
10        a = b;
11        b = nextTerm;
12    }
13 }

```

Explanation: Variables a and b track the two preceding sequence numbers. We print the current number, then calculate and shift the variables forward to generate the next number in the series.

Q20. Write a program to check if a given number is a Palindrome. A palindrome number reads the same forwards and backwards.

Sample 1 — Input: 121 $\Rightarrow$ Output: Palindrome

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int n, temp, rev = 0;
5      cin >> n;
6      temp = n;
7      while(temp > 0) {
8          rev = rev * 10 + (temp % 10);
9          temp /= 10;
10     }
11     if(rev == n) cout << "Palindrome" << endl;
12     else cout << "Not a Palindrome" << endl;
13 }

```

Explanation: By continually extracting the last digits mathematically and appending them to a new number (rev), we reverse the input and check for exact equivalence.

Q21. Take an integer K as input. Two variables a and b can take any integer value from 1 to K . Write a program to print all possible ordered combinations of (a, b) on a new line.

Sample — Input: 2 $\Rightarrow$ Output: 1, 1

1, 2...

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int K;
5      cin >> K;
6      for(int a = 1; a <= K; a++) {
7          for(int b = 1; b <= K; b++) {
8              cout << a << ", " << b << endl;
9          }
10     }
11 }

```


Explanation: A nested loop structure acts as a grid mapping system. The outer loop freezes at a number while the inner loop rapidly cycles through all its possibilities.

Q22. Write a program that prints all integers from 1 to 500 that contain the digit 2, the digit 7, or both, anywhere within their numerical representation.

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     for(int i = 1; i <= 500; i++) {
5         int temp = i;
6         bool found = false;
7         while(temp > 0) {
8             int digit = temp % 10;
9             if(digit == 2 || digit == 7) {
10                 found = true;
11                 break;
12             }
13             temp /= 10;
14         }
15         if(found) cout << i << " ";
16     }
17 }

```

Explanation: We deconstruct each number individually using a while loop to search for target digits, using a boolean flag to mark a successful hit.

Q23. Write a program to iterate through the numbers from 1 to 1000 and print only those numbers whose sum of digits is exactly 17.

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     for(int i = 1; i <= 1000; i++) {
5         int temp = i, sum = 0;
6         while(temp > 0) {
7             sum += (temp % 10);
8             temp /= 10;
9         }
10        if(sum == 17) cout << i << " ";
11    }
12 }

```

Explanation: Similar to previous problems, each number is dismantled. Here, we aggregate the digits and check if the sum satisfies the rigid criteria of being exactly 17.

Q24. Write a program that takes two positive integers as input and prints all the prime numbers that lie strictly between them.

Sample 1 — Input: 10 20 $\Rightarrow$ Output: 11 13 17 19

```

1 #include <iostream>
2 #include <cmath>

```

```

3 using namespace std;
4 int main() {
5     int a, b;
6     cin >> a >> b;
7
8     // swap if a is larger than b
9     if(a > b) {
10        int temp = a;
11        a = b;
12        b = temp;
13    }
14
15    for(int i = a + 1; i < b; i++) {
16        bool isPrime = true;
17        if(i <= 1) isPrime = false;
18
19        for(int j = 2; j <= sqrt(i); j++) {
20            if(i % j == 0) {
21                isPrime = false;
22                break;
23            }
24        }
25        if(isPrime) cout << i << " ";
26    }
27 }

```

Explanation: We first swap the boundaries manually. A prime number is only divisible by 1 and itself. Using the `sqrt()` function from the `<cmath>` library significantly reduces the loop checking time.

.....

Q25. Write a program that prints numbers starting from 1 up to 1000. However, the loop must terminate immediately as soon as it encounters a number whose sum of digits equals 23.

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     for(int i = 1; i <= 1000; i++) {
5         int temp = i, sum = 0;
6         while(temp > 0) {
7             sum += (temp % 10);
8             temp /= 10;
9         }
10        if(sum == 23) break;
11        cout << i << " ";
12    }
13 }

```

Explanation: Once the sum logic identifies a result equaling 23, the `break` keyword is deployed to kill the loop sequence entirely, ceasing further execution.

.....

Q26. Take an integer as input and mathematically subtract 293 from it. Compute and print the reversed value of this new resulting integer. Ensure your logic correctly handles negative

results.

Sample 1 — Input: 183 $\Rightarrow$ Math: $183 - 293 = -110 \Rightarrow$ Output: -11

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n;
5     cin >> n;
6     int res = n - 293;
7
8     int flag = 0;
9     // checking if res is negative
10    if(res < 0) {
11        flag = 1;
12        res = -res;
13    }
14
15    int rev = 0;
16    while(res > 0) {
17        rev = rev * 10 + (res % 10);
18        res /= 10;
19    }
20
21    if(flag == 1) {
22        rev = -rev;
23    }
24    cout << rev << endl;
25 }
```

Explanation: Reversing a negative number directly with standard math loops often causes bugs for beginners. We check if the result is negative using a simple `if` block, turn on a `flag` integer, calculate the absolute reversal, and re-apply the negative sign at the end if the flag is active.

.....

Q27. Write a program to print all uppercase English alphabets from 'A' to 'Z' along with their corresponding ASCII values using a loop.

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     for(char c = 'A'; c <= 'Z'; c++) {
5         cout << c << " - " << (int)c << endl;
6     }
7 }
```

Explanation: Because characters are inherently stored as ASCII integers in C++, we can iterate loops using `char` variables directly and cast them to `int` to output their numerical values.

.....

Q28. Write a program that accepts an integer input and finds the strictly next prime number that appears after it. If the user provides an input greater than 500, immediately print "Invalid Input" and terminate.

Sample 1 — Input: 14 $\Rightarrow$ Output: 17

```

1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4 int main() {
5     int n;
6     cin >> n;
7
8     if(n > 500) {
9         cout << "Invalid Input" << endl;
10    } else {
11        int curr = n + 1;
12        while(true) {
13            bool isPrime = true;
14            if(curr <= 1) isPrime = false;
15
16            for(int i = 2; i <= sqrt(curr); i++) {
17                if(curr % i == 0) {
18                    isPrime = false;
19                    break;
20                }
21            }
22
23            if(isPrime) {
24                cout << curr << endl;
25                break;
26            }
27            curr++;
28        }
29    }
30 }

```

Explanation: This code uses an if-else block to ensure inputs greater than 500 are handled without forcing an early function return. It then uses an infinite while loop to test successive integers (optimized using `sqrt()`), breaking out once a prime is confirmed.

.....

Q29. Take a positive integer input from the user and print its prime factors.

Sample — Input: 60 $\Rightarrow$ Output: 2 2 3 5

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n;
5     cin >> n;
6
7     for(int i = 2; i * i <= n; i++) {
8         while(n % i == 0) {
9             cout << i << " ";
10            n /= i;
11        }
12    }
13 }

```

```

14     if(n > 1) cout << n << endl;
15 }

```

Explanation: We systematically strip factors starting from 2. The inner `while` loop continuously extracts the same prime if applicable. Any value left over greater than 1 guarantees it is the final prime factor.

Q30. Write a program to compute a^b (a raised to the power b) where a is a decimal value and b is an integer entered by the user. Do **not** use the `pow()` function.

Sample 2 — Input: 2.5 -2 $\Rightarrow$ Output: 0.16

```

1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  int main() {
5      double a;
6      int b;
7      cin >> a >> b;
8
9      double res = 1.0;
10     int exp = abs(b);
11
12     for(int i = 0; i < exp; i++) {
13         res *= a;
14     }
15
16     if(b < 0) {
17         res = 1.0 / res;
18     }
19
20     cout << res << endl;
21 }

```

Explanation: To replicate exponent logic manually, we loop to multiply the base by itself relative to the absolute value of the exponent. If the exponent was originally negative, we mathematically invert the final result at the very end ($a^{-b} = 1/a^b$).

All the best! Keep Coding ♡